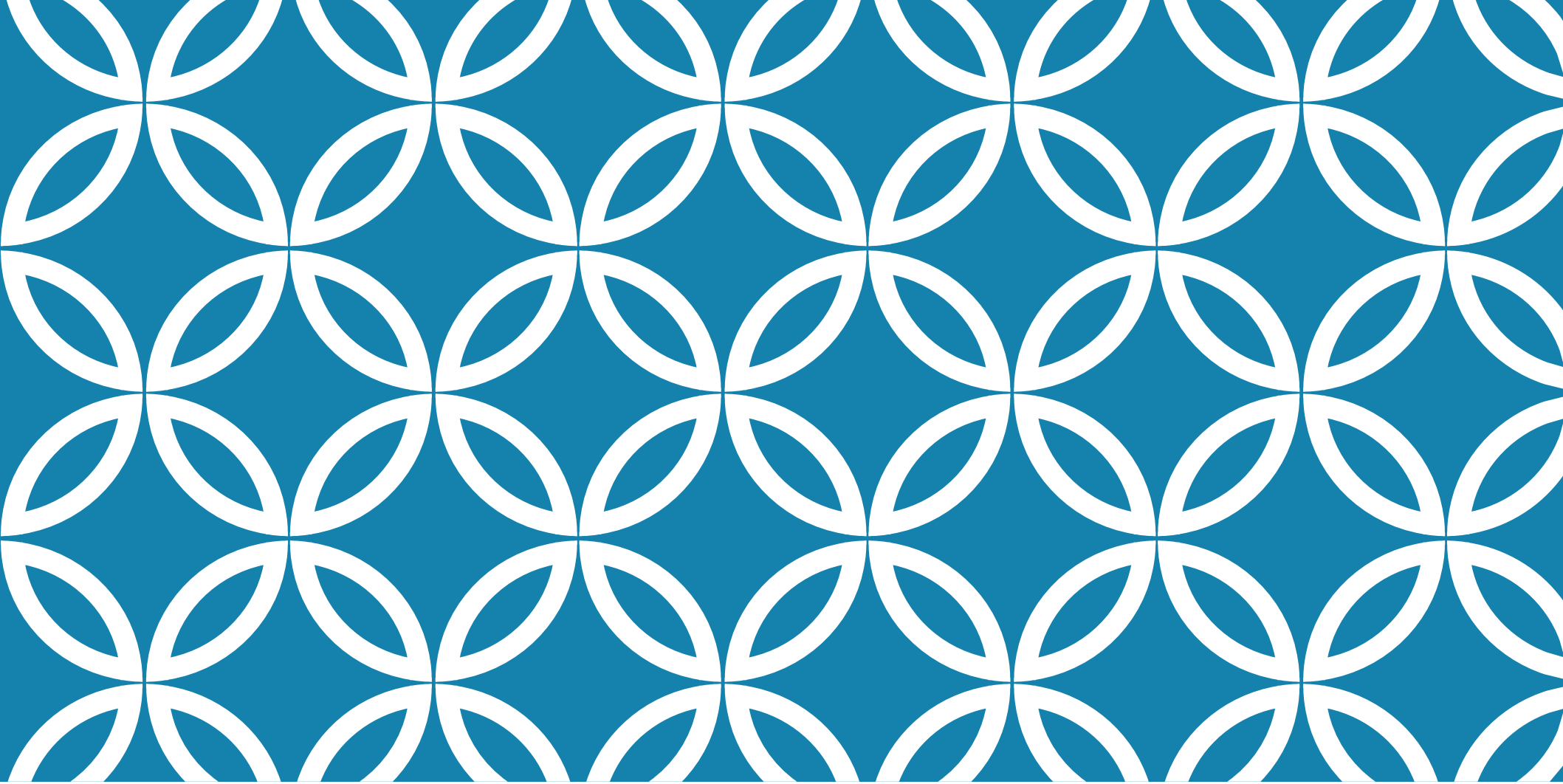


MORE ON GRAMMARS

AMBIGUITY IN GRAMMARS

Ambiguity implies the possibility of **different interpretation of a source string**

If a **grammar generates more than one parse tree for a string,** it is said to be an ambiguous grammar

Example

$E \rightarrow E + E \mid E * E \mid id$

String is **id + id * id**

Ambiguous grammar since we will have two parse trees

First parse tree

E → E + E

→ E + E * E

→ id + E * E

→ id + id * E

→ id + id * id

Second parse tree

E → E * E

→ E * E + E

→ id * E + E

→ id * id + E

→ id * id + id

ELIMINATING AMBIGUITY

1) Precedence

- Production at **higher levels** will have operators with **less priority**
- Production at **lower levels** will have operators with **higher priority**

2) Associativity

- **left to right,** prompt a **left recursion** in the production
- **right to left,** prompt a **right recursion** in the productions

E → E + E | E * E | id

We rewrite the grammar as below:

E → E + T | T

T → T * F | F

F → id

Now, this is an unambiguous grammar

E → E + T

→ T + T

→ F + T

→ id + T

→ id + T * F

→ id + F * F

→ id + id * F

→ id + id * id

EXERCISES

Find out if these grammars are ambiguous or not. If ambiguous then convert them into unambiguous grammar

1. E → E + E | E – E | id
2. E → E or E | E and E | not E | True | False
3. E → E + E | E * E | E ^ E | id

RECURSION

Two Types

1. Left recursion

2. Right recursion

Left Recursion

$A \rightarrow A\,\alpha \mid \beta$

Leads to **<u>Infinite loop</u>**

Generate the strings

$\beta\alpha$

$\beta\alpha\alpha\alpha\alpha$

ELIMINATE LEFT RECURSION

Make it right recursive

$A \rightarrow \beta\alpha^*$

Eliminating left recursion

$A \rightarrow \beta\ A'$

$A' \rightarrow \alpha\ A' \mid \varepsilon$

$E \rightarrow E + T \mid T$

After removing left recursion

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

EXERCISES

1. S → SA | B

2. S → (L) | x

   L → L, S | S

DETERMINISTIC AND NON-DETERMINISTIC GRAMMAR

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$

Suppose, derive the string $\alpha\beta_3$

Backtracking

NON-DETERMINISTIC GRAMMAR INTO DETERMINISTIC GRAMMAR

$$A \rightarrow \alpha\ \beta_1 \mid \alpha\ \beta_2 \mid \alpha\ \beta_3$$

Use Left Factoring

$$A \rightarrow \alpha\ A'$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$$

S → AB | AC

B → b

C → c

Using left factoring for S → AB | AC

S → AS’

S’ → B | C

B → b

C → c

EXERCISES

1. S → iEtS | iEtSeS | a
2. S→ aSSbS | aSaSb | abb | b